Appendix E — RAG, simply

Source: `RAG_SIMPLE.md`

RAG, explained simply

1. Chunking — cutting documents into pieces

Why chunk at all? You can't stuff a whole 50-page playbook into a prompt for every question. You break documents into small pieces ("chunks"), and only pull in the few pieces that are actually relevant to the question.

The three ways to cut:

- **Fixed-size** — just cut every N characters, no matter what's there. Fast but dumb: it'll slice a sentence, a table, or a heading right in half.
- **Recursive character split** (what you actually use) — try to cut at a paragraph break first; if a piece is still too big, try a line break; then a word break; then, as a last resort, mid-word. It *prefers* clean breaks over hitting the exact size limit.
- **Structural / semantic** — cut on the document's own structure (Markdown headers, HTML sections) or on meaning (find where the topic actually shifts). Best quality, more expensive to compute.

Why 800 characters, not 200 or 2000?

- Too small → a chunk loses context. "Standard discount is 15%" means nothing without knowing it's about the Enterprise tier.
- Too big → you dilute the chunk's meaning. One chunk covering three unrelated topics turns into a blurry "average" vector that doesn't strongly match *any* of the three — this is the single most common silent RAG failure.
- 800 characters ≈ 2–4 sentences: big enough to stand alone, small enough to stay about one thing.

Why overlap 40, not 0? A fact sitting right on a chunk boundary would otherwise be invisible to both chunks — overlap gives it a second chance to be captured whole in at least one of them. 40 characters ≈ one sentence fragment, and it only costs about 5% more storage.

Answer to give: "*I default to a recursive splitter at 800/40 — big enough for a chunk to be self-contained, small enough that its embedding has one dominant meaning. Then I measure recall@k on a real query set and adjust from there — it's a starting point, not a derived optimum.*"

---

2. Embeddings — turning text into a point in space

An embedding is just a list of numbers (a vector) that represents the *meaning* of a piece of text. Texts with similar meaning get vectors that point in similar directions. That's it — the individual numbers mean nothing on their own; only their relative position to other vectors matters.

Cosine similarity is just "how similar is the *direction* of two vectors, ignoring their length?" Score ranges from -1 (opposite meaning) to +1 (same meaning), 0 = unrelated. We ignore length (magnitude) on purpose — a long document and a short document can mean the same thing; we don't want length to skew the score.

The practical gotcha: in real embeddings, everything in a related topic scores somewhere between 0.7 and 0.999 — the differences are tiny (like 0.9981 vs 0.9979). So don't trust the raw number as "is this relevant" — trust the

ranking (which one's higher), not the absolute score. This is exactly why a reranker (section 5) matters — it's a second model that can actually tell things apart.

Dimension count (256 / 512 / 1536 / 3072): more dimensions = better at telling near-identical things apart, but more storage and slower search. 512 is a common sweet spot — modern embedding models are trained so the *most important* information lives in the first N numbers, so you can safely cut a 1536-dim vector down to its first 512 and lose very little quality (this trick is called Matryoshka embeddings).

Answer to give: "*512 dims isn't a weaker model — it's a truncated vector from a model trained so the important information front-loads into the first dimensions. I get a third of the storage and compute cost for a small, measured recall hit.*"

---

3. Vector index (HNSW) — how search stays fast at scale

The problem: comparing a query to every single one of 10 million vectors is way too slow.

The trick (HNSW): build a graph where each vector is connected to its nearest neighbors, arranged in layers — think of it like a highway system: top layer = a few long-distance highways that get you to roughly the right *region* fast, bottom layer = every local street for the precise final search. Searching means: start on the highway, zoom to the right area, then drop down through the layers, refining as you go.

Why it's "approximate": the greedy walk down the graph can get stuck in a local dead-end that *looks* close but isn't the true best match. You mitigate this by widening the "beam" of candidates you track (`ef_search`) — wider beam = better odds of finding the true best, but slower.

The two knobs that matter:

- `M` (built into the index) — how many neighbors each vector connects to. More connections = better recall, more memory, and it's baked in at build time (changing it means rebuilding the whole index).
- `ef_search` (set per query) — how wide a beam you search with. You can crank this up for an overnight accuracy check and turn it down for a fast interactive query, without touching the index.

Answer to give: "*HNSW for a mutable, latency-sensitive corpus — most enterprise RAG. IVF/IVF-PQ when the corpus is huge and mostly static and you're memory-bound. Either way, recall is something you measure on a golden query set, not something you assume from the index type.*"

---

4. Hybrid search — combining "meaning" search with "exact word" search

Two retrieval styles, two different blind spots:

- **Dense (embeddings)** — great at *meaning*: finds "quarterly revenue target" when you search "Q4 number." Terrible at exact tokens like a product SKU or an opportunity ID — those barely embed to anything meaningful.
- **Sparse (BM25 — classic keyword search)** — the opposite: exact on rare, specific words, blind to synonyms.

Since real users ask both kinds of question, you run **both** and combine the results.

BM25 in plain terms: score a document higher if it contains the query's words, but —

- **diminishing returns on repeats** — a document that says "Acme" 50 times isn't 50x more about Acme than one that says it once (keyword stuffing doesn't win).
- **rare words count more** — matching on "SOC2" (rare) is worth way more than matching on "the" (everywhere).

- **penalize long documents** — a long doc naturally contains more words by accident, so it's normalized against the average document length.

Combining the two ranked lists — Reciprocal Rank Fusion (RRF): you can't directly compare a BM25 score to a cosine score — different scales entirely. So RRF ignores the scores and only looks at *rank position* (1st, 2nd, 3rd...) in each list, and adds up `1/(60 + rank)` from each retriever. A document that ranks decently in *both* lists beats a document that's #1 in one list but completely absent from the other — it rewards consensus over one retriever's strong opinion.

Answer to give: "*Hybrid isn't sophistication for its own sake — dense and sparse retrieval fail on different query types, and in a real corpus you get both types of query in the same session. RRF combines them without needing to normalize incomparable scores, and it naturally favors documents both retrievers agree on.*"

---

5. Reranking — a second, smarter (but slower) pass

Bi-encoder (what your vector index uses): query and document are each turned into a vector *separately*, then compared. This is what makes it fast — you can precompute every document's vector ahead of time and just compare against the query at search time.

Cross-encoder (what a reranker is): query and document are fed into the model *together*, so it can actually reason about how they relate to each other — e.g. it can tell "the deal did NOT close" is a bad answer to "did the deal close," which a bi-encoder's blurry averaged vector often can't. The cost: nothing can be precomputed, so it's only practical on a small shortlist (tens of candidates), not millions.

So you use both, in two stages:

```
10M docs → [fast bi-encoder search] → top 50 candidates → [slow cross-encoder rerank] → top 5 → prompt
```

Stage 1's job is "don't lose the right answer" (recall). Stage 2's job is "put the right answer on top" (precision).

Answer to give: "*Bi-encoder search optimizes recall — don't lose the document. Cross-encoder reranking optimizes precision — get it to the top of a small candidate set. Skip reranking when your retrieval set is already small and clean (e.g. ACL-narrowed to 40 docs) — there's nothing to gain.*"

---

6. How you actually measure "is retrieval good?"

Always evaluate **retrieval separately from the final answer** — otherwise you can't tell if a bad answer came from "didn't find the right document" or "found it but didn't use it well."

- **Recall@k** — of all the documents that *should* have been found, what fraction actually made it into your top k? This is your ceiling: if the right document never gets retrieved, no amount of good generation can save you.
- **Precision@k** — of what you *did* return, what fraction was actually relevant? Irrelevant context isn't free — it burns tokens, dilutes the model's attention, and can carry a prompt-injection payload along for the ride.
- **MRR (Mean Reciprocal Rank)** — how far down the list was the *first* correct answer? Use this when there's one right answer and its position matters.
- **nDCG@k** — like recall/precision, but handles "how relevant" (not just yes/no) and cares more about getting the top few spots right than the bottom ones (matches how users actually pay attention — a lot at rank 1-2, way less by rank 9-10).
- **Leakage rate** (the one people forget) — how often did retrieval return something the caller wasn't allowed to see? Target: zero, always. This is a release blocker, not a "quality" metric — and volunteering this metric at a security company is a strong signal.

Answer to give: "*When recall drops, the first question is whether the retriever regressed or the corpus drifted stale — those have completely different fixes, and the metric alone doesn't tell you which. I track index freshness as a separate signal for exactly that reason.*"

---

7. Access control (ACL) filtering — the security-critical part

Post-filter (wrong): search everything, get your top-k, *then* throw away results the caller isn't allowed to see.

Pre-filter (right): bake the caller's permissions into the search itself, so restricted documents are never even candidates.

Why post-filter is a real leak, not just inefficient:

1. The restricted content already entered the working set — if there's a reranker or summarizer downstream, the model has already *read* the restricted text before the filter ever runs. "It wasn't shown to the user" doesn't undo that.
2. If someone normally gets 5 results but suddenly gets 2, they've just learned "3 things exist here that I can't see" — repeated queries turn this into a way to fingerprint what restricted content exists.
3. It relies on application code remembering to filter correctly, every time — a bug away from a leak. Pre-filtering pushes authorization into the query itself (or better, into the access token the retrieval service was given), so leakage is impossible *by construction*, not just "prevented by an if-statement."

And the sting in the tail: restricted documents are often the *most* relevant ones (the detailed internal version scores higher than the sanitized public one) — so post-filtering fails exactly on the queries where it matters most, and it also gives you a *worse* answer (fewer results) at the same time it leaks.

Answer to give: "*Authorization has to be part of the retrieval query, not a filter bolted on after. Post-filtering both leaks information through the shortfall itself and still exposes restricted content to anything downstream that reads the candidate set before the filter runs.*"

Cheat sheet — one line each

Topic	The one thing to remember
Chunking	Prefer clean semantic breaks over hitting an exact size; 800/40 recursive is a good, *measured* default.
Embeddings	Only relative direction (cosine) matters; absolute scores are nearly meaningless — trust rank, not value.
HNSW	Coarse-to-fine graph search; `ef_search` is your runtime recall/latency dial, `M` is baked in at build time.
Hybrid search	Dense finds meaning, BM25 finds exact tokens; RRF fuses by rank, not score, so it rewards consensus.
Reranking	Bi-encoder = fast + precomputable (recall). Cross-encoder = slow + sees both together (precision). Two stages.
Eval	Recall = ceiling, precision = noise cost, nDCG = position-aware quality, leakage rate = security release-blocker.
ACL	Authorization must be *in* the query, not a filter after — post-filter leaks by design, not by accident.
